The Core Concept: Docker Compose Does Everything

In this project, you are **NOT** doing this:

- ❌ Installing PostgreSQL on your machine
- ❌ Opening pgAdmin and creating tables manually
- ❌ Installing Milvus, MinIO, Airflow one by one
- ❌ Configuring each service to talk to each other

Instead, you write **one file** — docker-compose.yml — and run **one command**:

```
docker-compose up
```

This single command starts **ALL 7 services** automatically, creates networks between them, and they can talk to each other instantly.

What Are the 7 Services in Your Project?

Let me explain each one and **why it exists**:

Service 1: PostgreSQL 16 + pgvector

What it is: Your main database — stores all 20 tables (users, jobs, skills, recommendations, etc.)

Why not MySQL? Because your project needs pgvector — a PostgreSQL extension that stores **vector embeddings** directly inside the database. MySQL cannot do this. When you run docker-compose up, Docker automatically:

1. Downloads the PostgreSQL 16 image with pgvector pre-installed
2. Creates a database called job_recommendation_db
3. Sets the username/password from your .env file
4. Exposes it on port 5432

You connect to it like any normal database — but you never installed anything on your machine.

```
# Inside docker-compose.yml
postgres:
 image: pgvector/pgvector:pg16
 container_name: job_rec_postgres
 environment:
  POSTGRES_DB: job_recommendation_db
  POSTGRES_USER: ${DB_USER}
  POSTGRES_PASSWORD: ${DB_PASSWORD}
 ports:
  - "5432:5432"
 volumes:
  - postgres_data:/var/lib/postgresql/data
```

The volumes line is critical — it means even if you stop Docker, your data survives. Without this, every restart would erase your database.

Service 2: Milvus Lite (Vector Database)

What it is: A specialized database ONLY for vector similarity search.

Why do you need it when pgvector already exists? Think of it this way:

- **pgvector** → stores embeddings alongside your relational data (good for small-medium scale, filtering with SQL)
- **Milvus** → purpose-built for searching millions of vectors at high speed (good for the "find top 100 similar jobs" step)

In the matching pipeline, the flow is: Milvus finds the **top 100 candidate jobs** by vector similarity → then PostgreSQL applies detailed SQL filters (location, salary, experience) to narrow it to **top 20**.

```
milvus:
  image: milvusdb/milvus:v2.3-latest
  container_name: job_rec_milvus
  ports:
    - "19530:19530"   # gRPC port
    - "9091:9091"     # Health check
  volumes:
    - milvus_data:/var/lib/milvus
```

Service 3: MinIO (Object Storage)

What it is: A self-hosted S3-compatible file storage system.

Why? When a candidate uploads a resume (PDF, DOCX), you don't store the file inside PostgreSQL. That would be slow and bloated. Instead:

1. The raw file goes to **MinIO** (like Amazon S3, but free and local)
2. PostgreSQL only stores the **file path/URL** pointing to MinIO
3. When the AI agent needs to parse the resume, it fetches the file from MinIO

```
minio:
  image: minio/minio:latest
  container_name: job_rec_minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD}
  ports:
    - "9000:9000"   # API port
    - "9001:9001"   # Web console (you can see files in browser)
  volumes:
    - minio_data:/data
```

You can open http://localhost:9001 in your browser and visually see all uploaded resumes — just like AWS S3.

Service 4: Apache Airflow (Orchestration)

What it is: The **scheduler** that runs all your 10 DAGs (batch pipelines) on a 6-hour cycle.

Why? Your system is **batch-based**, not real-time. Every 6 hours, Airflow automatically:

- Processes new resume uploads → triggers ResumeProcessingGraph
- Ingests new jobs → triggers JobProcessingGraph
- Runs matching → triggers MatchingGraph
- Analyzes skill gaps → triggers GapAnalysisGraph
- Generates learning paths → triggers LearningPathGraph

Airflow itself needs 3 sub-components:

```
airflow-webserver:   # The UI you see at localhost:8080
airflow-scheduler:   # The brain that triggers DAGs on schedule
airflow-init:        # One-time setup (creates admin user, initializes DB)
```

All three run as separate containers but share the same configuration.

Service 5: Ollama (Local LLM Inference) — Production

What it is: Runs LLMs (Llama-3-8B, Mistral-7B) locally on your machine — no API calls, no internet needed.

During development, you skip this and use Groq API instead (faster, no GPU needed). During production, Ollama takes over.

```
ollama:
 image: ollama/ollama:latest
 container_name: job_rec_ollama
 ports:
  - "11434:11434"
 volumes:
  - ollama_data:/root/.ollama
 deploy:
  resources:
   reservations:
    devices:
     - capabilities: [gpu]   # Uses your NVIDIA GPU
```

Service 6: FastAPI (Your Application)

What it is: The Python backend — all REST API endpoints live here. Candidates upload resumes, recruiters search candidates, the frontend talks to this.

```
api:
  build: ./api
  container_name: job_rec_api
  ports:
    - "8000:8000"
  depends_on:
    - postgres
    - milvus
    - minio
  environment:
    DATABASE_URL:
postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/job_recommendation_db
    MILVUS_HOST: milvus
    MINIO_ENDPOINT: minio:9000
```

Notice depends_on — Docker ensures PostgreSQL, Milvus, and MinIO are running BEFORE FastAPI starts. This is the dependency order enforced automatically.

Service 7: Redis (Optional — for Airflow)

Airflow uses Redis as a message broker for its Celery executor (task queue). This is what allows Airflow to run multiple tasks in parallel.

How Services Talk to Each Other

This is the key insight. Inside Docker Compose, all services share a private network. They refer to each other by service name, not by IP address:

From	To	Connection String
FastAPI	PostgreSQL	postgresql://user:pass@postgres:5432/db
FastAPI	Milvus	milvus:19530
FastAPI	MinIO	minio:9000
Airflow	PostgreSQL	postgresql://user:pass@postgres:5432/db
LangGraph Agents	Ollama	http://ollama:11434
LangGraph Agents	Groq (dev)	https://api.groq.com (external)

The .env File — Single Source of Configuration

All secrets and settings live in ONE file:

```
# .env

DB_USER=jobrecuser

DB_PASSWORD=securepassword123

DB_NAME=job_recommendation_db


MINIO_USER=minioadmin

MINIO_PASSWORD=minioadmin123


GROQ_API_KEY=gsk_xxxxxxxxxxxx

GROQ_MODEL_PRIMARY=llama-3.1-8b-instant


AIRFLOW_ADMIN_USER=admin

AIRFLOW_ADMIN_PASSWORD=admin123
```

Docker Compose reads this file automatically and injects values into every service.

The Build Order — What Happens When You Run `docker-compose up`

```

Step 1: Docker pulls all images (postgres, milvus, minio, airflow, ollama)

Step 2: Creates a shared network (job_rec_network)

Step 3: Creates persistent volumes (postgres_data, milvus_data, minio_data, ollama_data)

Step 4: Starts PostgreSQL first (other services depend on it)
```

Step 5: Starts Milvus, MinIO in parallel

Step 6: Starts Airflow (webserver + scheduler)

Step 7: Starts FastAPI (waits for postgres, milvus, minio)

Step 8: All services are running and connected ✅

```

**Total time: ~2-5 minutes** on first run (downloading images), ~30 seconds on subsequent runs.

---

## Your Project Folder Structure

```

/job-recommendation-engine
├── docker/
│   └── docker-compose.yml   ← The ONE file that defines everything
├── .env   ← All secrets and config
├── db/
│   ├── migrations/   ← SQL files to create 20 tables
│   └── seeds/   ← Skill taxonomy seed data
├── api/
│   └── fastapi/   ← Python API code
├── agents/
│   └── langgraph/   ← All 5 LangGraph workflows
├── airflow/
│   └── dags/   ← All 10 DAG definitions
└── docs/
    └── specification.md

So to summarize — **you never touch MySQL, never install PostgreSQL, never manually create tables**. Docker Compose creates the entire infrastructure with one command. The database tables get created through **migration scripts** (SQL files in db/migrations/) that run automatically when the system starts.

What's your next question? Shall we go into how the **database migrations** work (how those 20 tables actually get created inside the Docker PostgreSQL)?